Which Tasks Tell, Which Tools Help: IRT-Guided Agent Benchmarking

Yiheng Yao
Stanford University
yaoyh@stanford.edu

Abstract

Agent benchmarks usually report aggregate pass rates, but those scores hide large variation in which tasks distinguish strong and weak model–harness systems, and which tool affordances make those differences possible. This project treats agent benchmark outcomes as measurement data. Using response matrices from the Holistic Agent Leaderboard (HAL), we model binary task outcomes with multidimensional item response theory (IRT), estimate task difficulty and discrimination, and study whether trace-derived harness features explain systematic residual structure. The central question is whether a smaller high-information subset of tasks can preserve useful ranking and calibration signal while also revealing which harness tools help which models on which kinds of items.

1 Introduction

Modern agent evaluations are expensive because each new model–harness combination may require running hundreds or thousands of tasks. They are also statistically blunt: an aggregate benchmark score treats every item as equally informative, even though some tasks are solved by nearly every agent, some by almost none, and only a subset separates systems in ways that are useful for model selection or harness design. This creates two related measurement problems. First, we need to know which benchmark items carry high information about agent capability. Second, we need to know whether performance differences come from the base model, the harness, the benchmark domain, or interactions among models, tools, and task types.

The research question is: *can IRT models fit to historical agent benchmark outcomes identify high-value tasks that preserve evaluation signal at lower cost, and can trace-derived harness features help explain which tools help which models on which kinds of items?*

Our approach differs from standard leaderboard analysis in three ways. First, we treat benchmark outcomes as a sparse item-response matrix rather than as aggregate pass rates. Second, we treat an evaluated agent as a model–harness system: the same base model may behave differently under different scaffolds, tools, prompts, and execution policies. Third, we use item-level residual structure to motivate tool and harness diagnostics, rather than assuming that benchmark-level labels such as "SWE-bench" or "GAIA" are sufficiently granular measurement units.

This project leverages the collected model $\times$ harness $\times$ benchmark dataset introduced by HAL [Kapoor et al., 2026]. The version analyzed here spans 59 recorded model labels and 68 recorded harness/scaffold labels, which we group into 20 semantic base-model groups and 15 semantic harness groups, across 10 benchmarks. Model-call settings such as reasoning effort are treated as model-configuration differences rather than separate harnesses. This normalization supports the paper's three analyses: item-level IRT for measurement, discrimination-based task downscaling for cheaper evaluation, and harness-aware residual analysis for generating targeted tool ablation hypotheses.

Benchmark	Tasks	Run configs	Norm. models	Norm. harnesses	Outcomes
assistantbench	33	33	10	2	1,089
colbench_backend_programming	1,000	10	7	1	10,000
corebench_hard	45	66	17	3	2,970
gaia	165	37	13	2	6,104
online_mind2web	336	36	12	2	10,800
scicode	65	38	11	3	2,470
scienceagentbench	102	25	12	2	2,550
swebench_verified_mini	50	45	15	2	2,147
taubench_airline	50	47	12	3	2,334
usaco	307	15	10	2	4,605

Table 1: Dataset coverage after regenerating the master binary outcome CSV and joining normalized metadata. The full matrix contains 45,069 observed outcomes across 10 benchmarks.

2 Background and Related Work

This project sits at the intersection of AI measurement science, item response theory, efficient evaluation, and process-level agent evaluation. Recent AI evaluation work argues that benchmarks should be understood as measurement instruments tied to explicit constructs and claims [Wallach et al., 2025, Salaudeen et al., 2025, Sun et al., 2025]. We adopt that stance for agent harnesses: the target construct is not "general intelligence," but the ability of model–harness systems to solve agentic benchmark items under particular tool affordances.

Item response theory was originally developed for educational testing, where the goal is to infer a respondent's latent ability from responses to items with varying difficulty and discrimination. The Rasch model and two-parameter logistic models provide a natural formalism for separating respondent ability from item difficulty and item informativeness [Rasch, 1960, Baker, 2001]. The same latent-variable view is closely related to probabilistic ranking models such as Bradley–Terry and Plackett–Luce models [Bradley and Terry, 1952, Luce, 1959], which infer strengths or utilities from noisy observations. In this project, agents replace students, benchmark tasks replace exam items, and binary pass/fail outcomes replace exam responses.

Recent LLM evaluation work has used IRT-style models to improve benchmark measurement and compression [Lalor et al., 2016, Zhou et al., 2026]. Truong et al.'s amortized model-based evaluation uses related structure to predict benchmark performance from partial item responses [Truong et al., 2025]. TinyBenchmarks similarly asks whether small curated subsets can recover full-benchmark estimates [Maia Polo et al., 2024], while Zhang et al. caution that such prediction can fail when new frontier models differ from historical systems [Zhang et al., 2025]. This tension motivates our emphasis on held-out prediction, calibration, ranking stability, and cold-start evaluation.

Agentic benchmarks make the measurement problem sharper because final correctness is only one view of tool-using behavior. TRAJECT-Bench evaluates whether tools are selected, parameterized, and ordered correctly [He et al., 2025]. AgentProcessBench provides human-labeled step annotations for tool-augmented trajectories [Fan et al., 2026]. HAL addresses the complementary infrastructure problem by standardizing agent evaluation across models, scaffolds, and benchmarks [Kapoor et al., 2026]. Our contribution is the statistical measurement layer on top of such data: estimating which items are informative and whether harness/tool features explain model–item residual structure.

3 Methods

Data. The primary data source is constructed from HAL benchmark outcomes. We use "agent configuration" to mean an observed model–harness run configuration: conceptually a base model paired with a harness/scaffold, and operationally identified in the extracted data by model, scaffold, run id, and benchmark. The regenerated master binary outcome file, `irt_data/response_matrix.csv`, contains 45,069 observed responses from 352 such configurations, corresponding to 222 distinct raw model–scaffold pairs, 165 normalized model–harness pairs, 59 recorded model labels, 68 recorded harness/scaffold labels, and 2,153 benchmark-task pairs. Because some raw model and harness labels include provider names, model names, or reasoning settings, Appendix B reports the normalization

Model terms	Features	Log loss	Pseudo-R^2	AUC
Benchmark	10	0.536	0.111	0.674
Benchmark + model + harness	45	0.521	0.136	0.715
+ model:harness + benchmark:harness	232	0.513	0.150	0.733
Task + model + harness	2,188	0.361	0.401	0.905
+ model:harness + benchmark:harness	2,375	0.346	0.427	0.913
+ task:harness	5,618	0.331	0.451	0.920

Table 2: Staged L2-regularized logistic audit. Pseudo-R^2 is McFadden-style deviance reduction relative to an intercept-only model. "Task" denotes task identity nested within benchmark.

from raw aliases to 20 semantic base-model groups and 15 semantic harness groups; reasoning effort is normalized onto the model-configuration side of the crossing rather than treated as a separate harness. Each row is an observed response $y_{ij} \in \{0, 1\}$ for agent configuration i on task j. The joinable metadata tables `irt_data/model_metadata.csv`, `irt_data/harness_metadata.csv`, and `irt_data/harness_affordances.csv` provide the canonical source of truth for normalized model labels, normalized harness labels, tool-exposure mechanisms, exposed-tool inventories, and binary harness affordance flags. Appendix C defines the tool-passage taxonomy, records the exposed tools for each normalized harness, and explains how the affordance flags are generated from harness source, configuration, and trace metadata.

3.1 Pseudo G-Study: Measurement Design Audit

Before fitting IRT, we run a lightweight generalizability-theory-inspired audit to decide where the measurement model needs resolution. The HAL matrix is sparse and unbalanced, and full crossed Bernoulli mixed models with high-cardinality interactions were computationally unstable. We therefore fit staged L2-regularized logistic models and compare McFadden-style pseudo-R^2, deviance, AUC, and residual structure (details in Appendix A):

$$y \sim \text{benchmark} + \text{model} + \text{harness} + \text{model} : \text{harness} + \text{benchmark} : \text{harness},$$
$$y \sim \text{task}(\text{benchmark}) + \text{model} + \text{harness} + \text{model} : \text{harness} + \text{benchmark} : \text{harness},$$
$$y \sim \text{task}(\text{benchmark}) + \text{model} + \text{harness} + \text{task} : \text{harness}.$$

This audit is not a causal tool analysis and does not estimate literal Gaussian variance components. It is a likelihood-based measurement diagnostic. If task identity and harness interactions materially reduce Bernoulli deviance, then aggregate benchmark scores are too coarse, and the subsequent model should be item-level and harness-aware.

3.2 Multidimensional IRT Model

The main model is a multidimensional two-parameter IRT model:

$$P(y_{ij} = 1) = \sigma(\theta_i^\top a_j + d_j),$$

where $\theta_i \in \mathbb{R}^K$ is the latent ability vector of agent configuration i, $a_j \in \mathbb{R}^K$ is the discrimination vector for task j, d_j is task easiness, and σ is the logistic function. We train by minimizing binary cross-entropy over observed agent–task outcomes. We sweep small values of K and compare latent-only models against feature-informed models in which agent and task metadata are projected into the same K-dimensional space and added to the learned latent embeddings. The fitted model supports two co-equal analyses: task discrimination for reducing benchmark size, and residual structure for identifying model–harness and harness–task interactions.

3.3 Harness Feature and Residual Model

The feature-informed model decomposes each agent configuration into model, harness, and task features. Model features include provider, normalized model family, and model-call configuration metadata such as reasoning effort where available. Harness features come from the normalized taxonomy in Appendix C: a categorical tool-passage mechanism, a concrete exposed-tool inventory, and binary affordance flags for code generation, code execution, shell access, file read/edit, web

search, page browsing, browser control, vision, native domain tools, benchmark API tools, human dialogue, retrieval context, external judging/evaluation, self-debug feedback, and complete-program prompting. Task features include benchmark domain and coarse task type.

We use these features in two ways. First, in the feature-informed IRT model, metadata provides a cold-start prior for configurations with weak or absent response history. We distinguish three cold-start information regimes: *different-harness*, where the model is known but evaluated through a harness not observed with that model in the benchmark; *open-code*, where source/configuration inspection gives harness affordance flags before any outcomes are observed; and *partially open-trace*, where a small trace subsample is available to refine affordance labels before predicting the remaining tasks. Second, after fitting the IRT model, we compute response residuals $r_{ij} = y_{ij} - \hat{p}_{ij}$ and aggregate them by normalized model family, harness, benchmark/task type, and affordance flags. This residual analysis asks whether particular tool-passage mechanisms or affordances are associated with systematic over- or under-performance after accounting for item difficulty and overall agent ability. These estimates are observational and hypothesis-generating: tool exposure is confounded with many harness design choices, and exposure does not guarantee effective tool use.

3.4 Task Selection and Adaptive Evaluation

After fitting the model, we rank tasks by discrimination magnitude $\|a_j\|$ within each benchmark. We construct fixed subsets that cover 50%, 70%, 80%, and 90% of total estimated discrimination mass. These subsets operationalize the question: how much of the benchmark's agent-separating signal can be preserved with fewer tasks? This connects the IRT model to active learning and sample-efficient measurement: the fixed subsets are a static approximation to adaptive testing policies that would select high-information items as evidence accumulates.

3.5 Evaluation Protocol

We evaluate predictive and ranking quality on a reproducible semantic holdout split. The leakage unit is benchmark $\times$ normalized model $\times$ normalized harness, since a row-level random split would place the same semantic model–harness unit in both train and test through raw aliases or repeated runs. For each benchmark $\times$ normalized harness cell, the split holds out one representative normalized model whose pass rate is closest to that cell's median, avoiding obvious cherry-picking of only unusually strong or weak systems. The resulting split contains 39,461 training rows and 5,608 holdout rows, with no leakage-unit overlap between train and holdout.

Metrics include binary cross-entropy, AUC, accuracy, Brier score, calibration error, rank correlation between full-suite and subset-based agent rankings, and stability of selected high-discrimination tasks under resampling. For the feature analysis, we compare latent-only and metadata-informed models on held-out semantic units. For cost-aware evaluation, we report task-count reduction at each discrimination-coverage threshold and the loss in predictive or ranking fidelity relative to the full benchmark.

4 Results and Discussion

4.1 Exploratory Measurement Audit

Table 2 reports the pseudo G-study on the regenerated 45,069-row response matrix. Benchmark labels alone reduce Bernoulli deviance only modestly relative to the intercept-only model ($R^2_{\text{pseudo}} = 0.111$). Adding normalized model and harness main effects raises this to 0.136, while model–harness compatibility raises it to 0.150. In contrast, replacing benchmark identity with task identity nested within benchmark produces a much larger jump ($R^2_{\text{pseudo}} = 0.401$). Adding model–harness and benchmark–harness interactions reaches 0.427, and a task–harness term reaches 0.451. This supports the paper's central modeling choice: benchmark-level aggregation is too coarse, and the useful resolution is item-level and harness-aware.

The regularized interaction coefficients also provide an initial qualitative check that harness effects are not scalar. HF Open Deep Research shows positive model–harness residual structure with GPT-5, o4-mini, GPT-4.1, and Claude Opus 4, but negative residual structure with Claude Opus 4.1 and Claude Sonnet 4.5. HAL Generalist shows the complementary pattern of stronger residual performance

K	Features	BCE	AUC	Accuracy	Brier	Cal. error
1	yes	0.358	0.895	0.835	0.113	0.030
2	yes	0.362	0.900	0.833	0.116	0.086
4	yes	0.376	0.896	0.832	0.117	0.072
8	yes	0.425	0.890	0.821	0.129	0.124
1	no	0.566	0.861	0.823	0.188	0.191
2	no	0.583	0.852	0.827	0.196	0.184
4	no	0.596	0.844	0.823	0.202	0.195
8	no	0.604	0.842	0.825	0.206	0.228

Table 3: Holdout performance on the split-v1 semantic holdout. The leakage unit is benchmark $\times$ normalized model $\times$ normalized harness. Lower BCE, Brier, and calibration error are better; higher AUC and accuracy are better.

with several Claude-family configurations and weaker residual performance with o3, Gemini, and DeepSeek variants. These are descriptive in-sample diagnostics, not causal tool effects, but they motivate the later held-out MIRT residual analysis of which harness mechanisms fit which model families and item types.

4.2 MIRT Model Comparison

Table 3 reports the semantic-holdout sweep. Because all holdout agent configurations are unseen under the split unit, latent-only models cannot estimate a held-out agent ability vector from response history. For this evaluation, unseen latent-only agents are scored with zero θ_i, so their predictions reduce mostly to task easiness. The feature-informed models also zero unseen learned embeddings, but retain the metadata projection $f_{\text{agent}}(x_i)$, giving a genuine cold-start agent vector without fitting on held-out responses. Feature-informed models therefore dominate latent-only models on holdout BCE and Brier score. The $K = 2$ feature-informed model is the best prediction model by holdout AUC, while the $K = 1$ feature-informed model gives the strongest benchmark-compression operating point.

There is also a prediction–reduction tradeoff across fitted models. At the 80% discrimination-coverage threshold, the $K = 2$ feature-informed model has the best holdout AUC (0.900) but removes only 20.6% of tasks on average. The $K = 1$ feature-informed model has slightly lower AUC (0.895) and the best holdout BCE (0.358), but its discrimination mass is more concentrated: it removes 46.2% of tasks on average at the same 80% coverage threshold. Thus the Pareto frontier contains at least two useful operating points: $K = 2$ for maximum ranking signal, and $K = 1$ for a more aggressive screening subset with only a small AUC loss.

4.3 Benchmark Downscaling

Using the compression-oriented $K = 1$ feature-informed model, we rank tasks within each benchmark by $|a_j|$ and compute the number of tasks needed to retain fixed fractions of total discrimination mass. Figure 2 shows the resulting downscaling curves. The fitted one-dimensional discrimination spectrum is substantially more concentrated than the higher-dimensional spectra: retaining 80% of discrimination mass keeps a mean of 53.8% of tasks across benchmarks, corresponding to a mean 46.2% task-count reduction. At 70% discrimination coverage, the mean task-count reduction is 55.6%; at 90% coverage, it is 32.1%. In aggregate task-count terms, the conservative 90% threshold retains 1,426 of 2,153 tasks and removes 727 tasks, corresponding to a 33.8% estimated benchmark-cost reduction. This gives the strongest screening contribution: the $K = 1$ feature model gives up only 0.004 holdout AUC relative to $K = 2$ with features while cutting nearly half of benchmark tasks at the 80% discrimination threshold and roughly one-third of tasks even at the conservative 90% threshold.

4.4 Which Tool Interfaces Help?

The strongest tool result is a negative one: broad code-agent tool registries often underperform benchmark-specialized interfaces in matched comparisons. Table 4 reports same-benchmark, same-model contrasts where the base model is held fixed and only the harness/tool interface changes. On

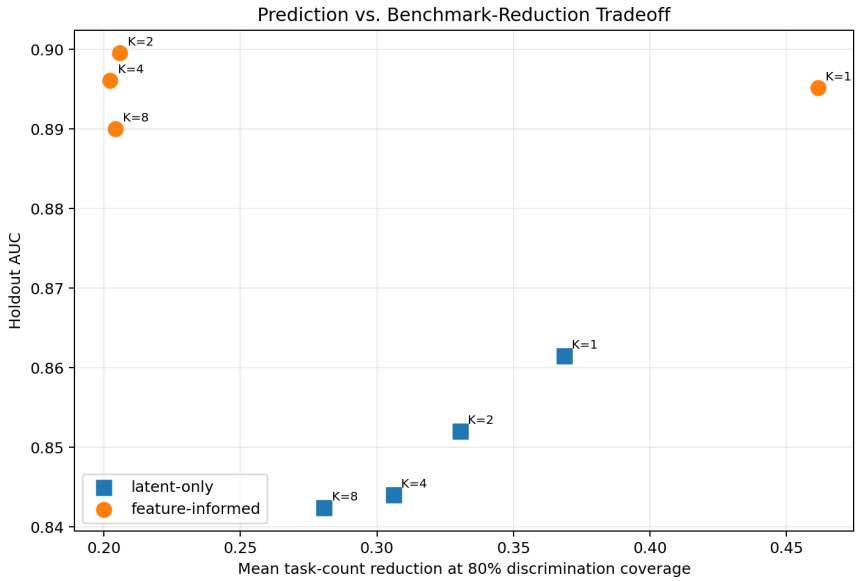


Figure 1: Tradeoff between holdout AUC and mean task-count reduction at the 80% discrimination-coverage threshold. Feature-informed $K = 1$ and $K = 2$ models form the useful frontier: $K = 2$ maximizes AUC, while $K = 1$ gives much larger task reduction with a small AUC decrease.

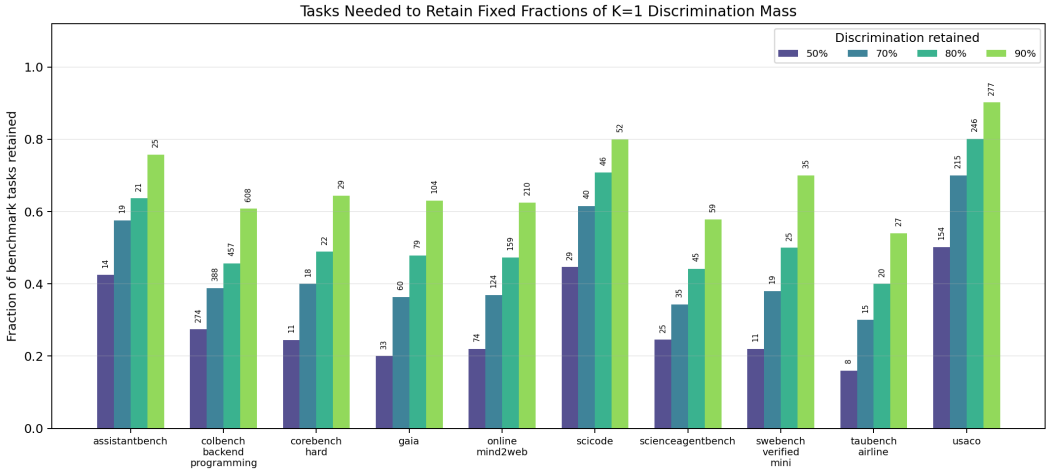


Figure 2: Fraction of tasks retained to preserve fixed fractions of total estimated discrimination mass under the compression-oriented $K = 1$ feature-informed MIRT model. Bar labels give the retained task count.

SWE-bench, SWE-Agent's repository workflow—file-map navigation, shell and editor commands, diff/review state, and explicit submit actions—substantially outperforms HAL Generalist's code-agent registry for every shared model shown. On TAU-bench Airline, native airline tool schemas outperform HAL Generalist's wrapped domain actions. On ScienceAgentBench, SAB Self-Debug outperforms HAL Generalist for the shared o3 run, suggesting that complete-program generation plus harness execution/debug feedback is better aligned with scientific coding tasks than a general-purpose registry loop.

The pattern suggests that "which tools help" is conditional on the task's latent work loop. Repository-editing affordances help software repair, native domain schemas help transactional customer-service tasks, and execution/debug feedback helps scientific coding. Conversely, simply exposing many tools through a broad code-agent registry is not enough: the interface must make the task-relevant workflow

Benchmark	Model	Interface contrast	Task-specific	HAL registry	Δ
SWE-bench	o3	SWE-Agent vs. HAL Generalist	0.593	0.000	0.593
SWE-bench	o4-mini	SWE-Agent vs. HAL Generalist	0.551	0.040	0.511
SWE-bench	GPT-5	SWE-Agent vs. HAL Generalist	0.640	0.140	0.500
SWE-bench	GPT-4.1	SWE-Agent vs. HAL Generalist	0.469	0.020	0.449
TAU-bench	GPT-4.1	TAU FewShot vs. HAL Generalist	0.560	0.160	0.400
TAU-bench	o4-mini	TAU FewShot vs. HAL Generalist	0.545	0.200	0.345
TAU-bench	o3	TAU ToolCalling vs. HAL Generalist	0.540	0.208	0.332
ScienceAgentBench	o3	SAB Self-Debug vs. HAL Generalist	0.333	0.098	0.235

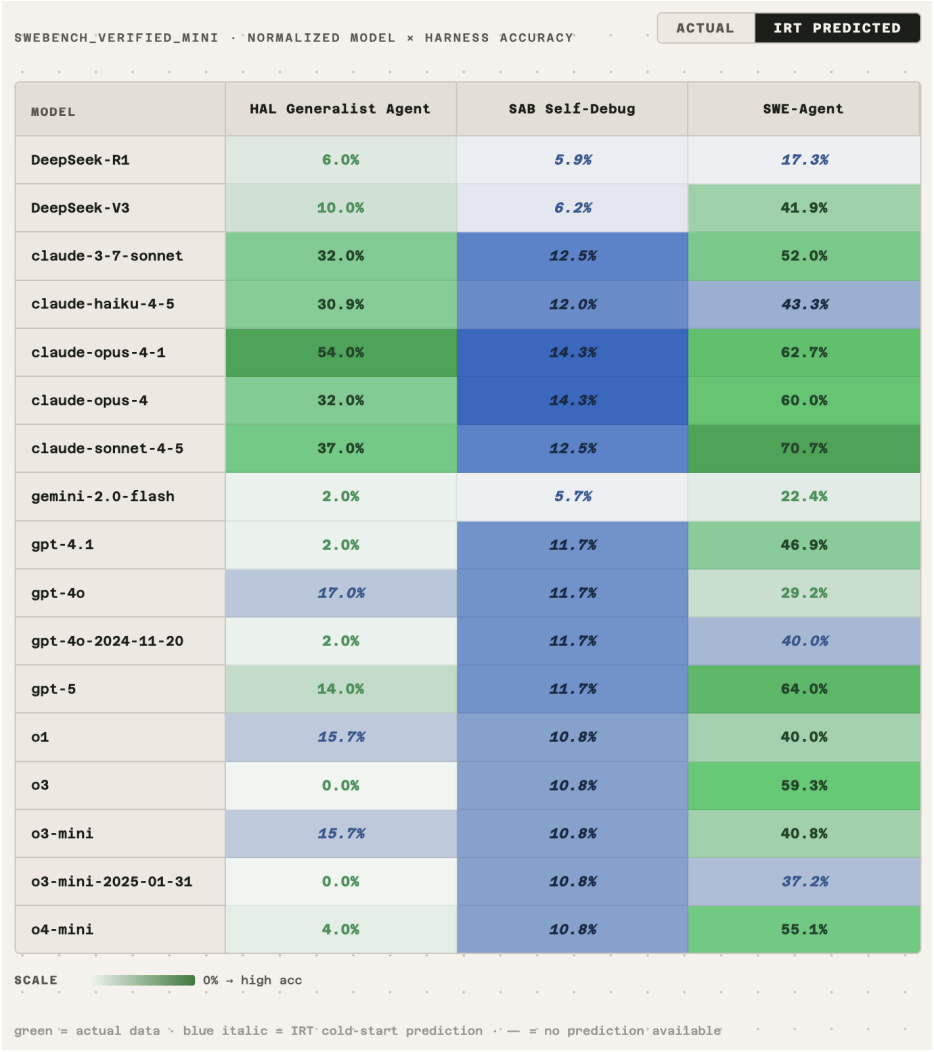
Table 4: Matched same-model, same-benchmark pass-rate contrasts. "Task-specific" means the benchmark-specialized interface, which may be a repo-repair workflow, native domain-tool schema, or execution/debug scaffold. "HAL registry" is HAL Generalist's code-agent tool registry. These observational contrasts motivate targeted ablations; they do not by themselves identify causal effects of individual tools.

easy for the model to represent and execute. A targeted tool-count check supports this interpretation: across matched benchmark $\times$ normalized-model configurations, the lower-tool-count harness wins 61.7% of comparisons, but the largest counterexamples are task-aligned native schemas such as TAU-bench. Appendix D audits matched SWE-bench traces and shows that HAL Generalist does have live execution and edit affordances; the stark SWE-bench gap therefore motivates interface-level ablations rather than a simple sandbox/no-sandbox explanation.

Model–harness heatmap. Figure 3 shows the demo heatmap view for SWE-bench Verified Mini, aggregating outcomes by normalized model and harness. Green cells are observed accuracies; blue italic cells are IRT cold-start predictions for model–harness pairs without observed runs in that benchmark slice. The view makes the main qualitative pattern visible: SWE-Agent's repository-specific workflow dominates the broad HAL Generalist registry on shared models, while the SAB Self-Debug column is an extrapolated prediction rather than a measured SWE-bench result.

Targeted ablation hypotheses. These contrasts suggest three concrete small-budget ablations: pass TAU-bench native schemas directly through HAL Generalist rather than through registry wrappers; add a SWE-Agent-like file-map/edit/review loop to HAL Generalist on SWE-bench, or disable pieces of that loop inside SWE-Agent; and run SAB with self-debug feedback disabled to separate complete-program prompting from execution-loop feedback. The goal is not to exhaustively optimize a harness, but to test whether the measurement model produces plausible interventions.

Limitations. The main limitations are sparsity, benchmark heterogeneity, non-random tool exposure, and dependence on the semantic split. A harness may expose a tool without using it well; conversely, a missing tool may proxy for many other scaffold design choices. The holdout split tests generalization across benchmark-specific model–harness units, but it does not prove performance on entirely unseen future models or unseen harnesses. We therefore treat tool-feature estimates as hypothesis-generating and rely on targeted ablations for stronger evidence.



MODEL	HAL Generalist Agent	SAB Self-Debug	SWE-Agent
DeepSeek-R1	6.0%	*5.9%*	*17.3%*
DeepSeek-V3	10.0%	*6.2%*	41.9%
claude-3-7-sonnet	32.0%	*12.5%*	52.0%
claude-haiku-4-5	30.9%	*12.0%*	*43.3%*
claude-opus-4-1	54.0%	*14.3%*	62.7%
claude-opus-4	32.0%	*14.3%*	60.0%
claude-sonnet-4-5	37.0%	*12.5%*	70.7%
gemini-2.0-flash	2.0%	*5.7%*	22.4%
gpt-4.1	2.0%	*11.7%*	46.9%
gpt-4o	*17.0%*	*11.7%*	29.2%
gpt-4o-2024-11-20	2.0%	*11.7%*	*40.0%*
gpt-5	14.0%	*11.7%*	64.0%
o1	*15.7%*	*10.8%*	40.0%
o3	0.0%	*10.8%*	59.3%
o3-mini	*15.7%*	*10.8%*	40.8%
o3-mini-2025-01-31	0.0%	*10.8%*	*37.2%*
o4-mini	4.0%	*10.8%*	55.1%

Figure 3: Demo heatmap for SWE-bench Verified Mini accuracy by normalized model and harness. Green cells are observed pass rates; blue italic cells are IRT cold-start predictions for unobserved model–harness combinations.

5 Conclusion

HAL outcomes are more informative when modeled as item-response data for model–harness systems than when reduced to aggregate leaderboard scores. In the 45,069-row matrix, task identity and task–harness interactions explain far more deviance than benchmark labels alone, motivating item-level IRT. On the semantic holdout, feature-informed MIRT gives a cold-start gain over latent-only models: $K = 2$ with features is the best prediction setting (AUC 0.900), while $K = 1$ with features is the best screening setting. At a conservative 90% discrimination-mass threshold it removes 727 of 2,153 tasks, a 33.8% aggregate cost reduction; at 80% it removes nearly half of tasks on average. The tool analysis points to a conditional answer to "which tools help": specialized, task-aligned interfaces—repository editing for SWE-bench, native schemas for TAU-bench, and execution/debug feedback for scientific coding—beat a broad code-agent registry in matched comparisons. The resulting ablation agenda is concrete: test native-schema passthrough, repo-workflow components, and self-debug feedback as targeted interventions rather than treating tool exposure as a monolithic capability.

6 AI Disclosure Statement

Addressing Plagiarism, Bias, and Inaccuracies. I first identified the relevant literature and key studies my project builds off from, then employed the use of perplexity in widening the search radius around other related works before finally drafting the literature review section of my final report. My project is mainly methodological with low risk of biases from LLM generated text. However, admittedly, cultural-linguistic biases in terms of missing out non-english academic literature remains a risk which I have not corrected for (e.g. searching for existing literature in Japanese, Chinese, and other languages).

How and why you used AI tools. I employed AI tools in 3 key areas: A) widening literature review scope. AI tooling can better target relevant sources by quickly consuming full text beyond just the title and abstract much faster than I can, in this regard, I have chosen to employ AI tools for widening literature review to better encompass and credit existing literature from which my study benefits from. B) data analysis scripting. Once the research questions are scoped out (manually by a human), the manual repetitive coding task of scripting data cleaning, processing, and figure generation I chose to defer to a coding agent. This process was nonetheless still tightly collaborative in my constant supervision of the coding agent employing the right data cleaning methodology, train-test-validation split, statistical modeling approaches, and leaving a record of data provenance such that the study is replicable. Final figures and resulting scripts were checked in and briefly scanned for incorrect assumptions but otherwise entirely AI generated. C) Refinement of prose and LaTeX formatting. I start drafting the final report by writing in a plain text editor in markdown then allowing a coding agent to transpile that into .tex format. This allows much quicker human input because I no longer needed to worry about the styling of the document and can focus better on the substantive portion of academic writing. Overall I use AI tooling where the path to achieving the end product is clear to me but AI can do it much faster than I can.

Code availability. The project code is available at `https://github.com/devYaoYH/hal-harness-plus-plus`. This repository is a fork of the public HAL harness; the substantial project-specific IRT data preparation, modeling, and analysis changes are located in `https://github.com/devYaoYH/hal-harness-plus-plus/tree/main/irt_data`.

Impact Statement. Cost reductions from employing a subset of Agentic benchmark tasks can help reduce the environmental impact of AI technology development. However, as the study notes, this subset is no longer as accurate as the original and there could be inflated risk in putting agents into production environments when tested on fewer tasks. Downstream inferences drawn from unvalidated predictions made by the models introduced in this study can also result in potential harmful consequences. As such, in this study I have refrained from presenting quantitative claims but rather directional and carefully guarded assertions. Furthermore a clear limitations section points readers to the speculative nature of the predictions generated and warns against taking it as causal evidence. To the extend I am aware, I have complied with Stanford's standards of academic integrity in my preparation of this report. No humans were involved in data collection or participatory trials for this study.

References

Frank B. Baker. *The Basics of Item Response Theory*. ERIC Clearinghouse on Assessment and Evaluation, 2001.

Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

Shengda Fan, Xuyan Ye, Yupeng Huo, et al. AgentProcessBench: Diagnosing step-level process quality in tool-using agents. *arXiv preprint arXiv:2603.14465*, 2026.

Pengfei He, Zhenwei Dai, Bing He, et al. TRAJECT-Bench: A trajectory-aware benchmark for evaluating agentic tool use. *arXiv preprint arXiv:2510.04550*, 2025.

Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin,

Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://arxiv.org/pdf/2510.11977.

John P. Lalor, Hao Wu, and Hong Yu. Building an evaluation scale using item response theory. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 648–657, 2016. doi: 10.18653/v1/D16-1062.

R. Duncan Luce. *Individual Choice Behavior: A Theoretical Analysis*. Wiley, 1959.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 34303–34326. PMLR, 2024. URL https://proceedings.mlr.press/v235/maia-polo24a.html.

Georg Rasch. *Probabilistic Models for Some Intelligence and Attainment Tests*. Danish Institute for Educational Research, 1960.

Olawale Salaudeen, Anka Reuel, Ahmed Ahmed, Suhana Bedi, Zachary Robertson, Sudharsan Sundar, Ben Domingue, Angelina Wang, and Sanmi Koyejo. Measurement to meaning: A validity-centered framework for AI evaluation. *arXiv preprint arXiv:2505.10573*, 2025.

Luning Sun, Christopher Gibbons, José Hernández-Orallo, Xiting Wang, Liming Jiang, David Stillwell, Fang Luo, and Xing Xie. Beyond benchmarks: Evaluating generalist medical artificial intelligence with psychometrics. *Journal of Medical Internet Research*, 27:e70901, 2025. doi: 10.2196/70901.

Sang Truong, Yuheng Tu, Percy Liang, Bo Li, and Sanmi Koyejo. Reliable and efficient amortized model-based evaluation. *arXiv preprint arXiv:2503.13335*, 2025.

Hanna Wallach, Meera Desai, A. Feder Cooper, Angelina Wang, Chad Atalla, Solon Barocas, Su Lin Blodgett, Alexandra Chouldechova, Emily Corvi, P. Alex Dow, Jean Garcia-Gathright, Alexandra Olteanu, Nicholas J. Pangakis, Stefanie Reed, Emily Sheng, Dan Vann, Jennifer Wortman Vaughan, Matthew Vogel, Hannah Washington, and Abigail Z. Jacobs. Position: Evaluating generative AI systems is a social science measurement challenge. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 82232–82251. PMLR, 2025. URL https://proceedings.mlr.press/v267/wallach25a.html.

Guanhua Zhang, Florian E. Dorner, and Moritz Hardt. How benchmark prediction from fewer data misses the mark. *arXiv preprint arXiv:2506.07673*, 2025.

Hongli Zhou, Hui Huang, Lvyuan Han, et al. Lost in benchmarks? rethinking large language model benchmarking with item response theory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(41):35085–35093, 2026. doi: 10.1609/aaai.v40i41.40814.

A Pseudo G-Study Methodology

Classical generalizability theory decomposes measurement variation across facets such as persons, items, raters, and occasions. In HAL, the analogous facets are base model, harness, benchmark, and task. A balanced random-effects G-study is not appropriate because the observed response matrix is sparse and highly unbalanced: not every model–harness pair is evaluated on every task.

We initially considered Bernoulli mixed-effects models of the form

$$y \sim 1 + (1|\text{task}) + (1|\text{model}) + (1|\text{harness}) \\ + (1|\text{model} : \text{harness}) + (1|\text{benchmark} : \text{harness}) + (1|\text{task} : \text{harness}).$$

This is the closest analogue to a G-study for binary outcomes, but the full Bayesian version was computationally unstable for the current matrix, especially with high-cardinality task–harness effects. We therefore use a staged L2-regularized logistic fixed-effect audit in the main analysis.

For each staged model, we compute Bernoulli log loss,

$$\ell = -\frac{1}{N}\sum_i \left[y_i \log p_i + (1-y_i)\log(1-p_i)\right],$$

and deviance,

$$D = 2N\ell.$$

We report McFadden-style pseudo-R^2,

$$R^2_{\text{pseudo}} = 1 - \frac{D_{\text{model}}}{D_{\text{null}}},$$

where the null model is intercept-only. This quantity should be interpreted as relative reduction in Bernoulli deviance, not as ordinary least-squares variance explained. Because all staged models are fit to the same binary rows and likelihood family, the comparisons indicate which facets materially reduce predictive uncertainty.

We do not use a Gaussian linear mixed model as the main audit because the outcome is binary. A linear probability mixed model can produce probabilities outside $[0, 1]$, has heteroskedastic residuals by construction, and estimates variance components on a scale that does not match the Bernoulli likelihood used by the IRT model. The staged logistic audit is therefore a more regular and reproducible diagnostic for this response matrix.

B Dataset Inventory

This appendix groups the benchmark, model, and harness/scaffold labels present in the regenerated training response matrix. The raw HAL metadata contains provider prefixes, date suffixes, reasoning-setting suffixes, and model names embedded in some harness labels. For modeling, we therefore treat the raw labels as aliases and normalize them into semantic model and harness groups. This is especially important for harnesses: labels such as "Core Agent Opus 4.5" describe a CORE-Agent run paired with a particular model, not a fundamentally separate harness. Likewise, "HAL Generalist High Reasoning" is normalized as the HAL Generalist harness paired with a model configuration that sets a reasoning/thinking budget.

B.1 Benchmarks

B.2 Semantic Model and Model-Configuration Groups

The 59 recorded model labels collapse to 20 semantic base-model groups. When a run sets reasoning_effort or an equivalent provider reasoning-token budget, we treat that as part of the model configuration rather than as a separate harness. Thus, for example, gpt-5 with high reasoning effort and gpt-5 with minimal reasoning effort are distinct model configurations paired with the same HAL Generalist harness. We keep the raw aliases visible so that the normalization remains auditable. The joinable model normalization table used for analysis is stored in `irt_data/model_metadata.csv`; it preserves each raw model label while recording the normalized base-model group and provider/family metadata.

- **Claude 3.7 Sonnet**: Anthropic Claude Sonnet model. Aliases: anthropic/claude-3-7-sonnet-20250219; anthropic/claude-3.7-sonnet; claude-3-7-sonnet-20250219; openrouter/anthropic/claude-3.7-sonnet; openrouter/anthropic/claude-3.7-sonnet:thinking.
- **Claude Haiku 4.5**: Anthropic Claude Haiku model. Aliases: anthropic/claude-haiku-4-5; claude-haiku-4-5; claude-haiku-4-5-20251001; openrouter/anthropic/claude-haiku-4.5.
- **Claude Opus 4**: Anthropic Claude Opus model. Aliases: claude-opus-4-20250514; openrouter/anthropic/claude-opus-4.
- **Claude Opus 4.1**: Anthropic Claude Opus model. Aliases: anthropic/claude-opus-4.1; claude-opus-4-1; claude-opus-4-1-20250805; openrouter/anthropic/claude-opus-4.1.

Benchmark	Brief description
assistantbench	Assistant-oriented web and information-seeking tasks evaluated with browser-capable agents.
colbench_backend_programming	Backend programming tasks from ColBench.
corebench_hard	Hard CORE-Bench tasks emphasizing complex tool-using problem solving.
gaia	General Assistant benchmark tasks requiring multi-step reasoning and external information use.
online_mind2web	Web navigation and interaction tasks derived from Mind2Web-style environments.
scicode	Scientific coding tasks requiring code synthesis and problem-specific reasoning.
scienceagentbench	ScienceAgentBench tasks evaluating scientific analysis and self-debugging workflows.
swebench_verified_mini	A compact verified SWE-bench subset for software-engineering issue resolution.
taubench_airline	TAU-bench airline customer-service tool-use tasks.
usaco	Competitive-programming tasks from the USACO-style coding environment.

Table 5: Benchmarks represented in the regenerated response matrix.

- **Claude Opus 4.5**: Anthropic Claude Opus model. Aliases: anthropic/claude-opus-4-5-20251101; claude-opus-4-5; claude-opus-4-5-20251101.
- **Claude Sonnet 4**: Anthropic Claude Sonnet model. Alias: claude-sonnet-4-20250514.
- **Claude Sonnet 4.5**: Anthropic Claude Sonnet model. Aliases: anthropic/claude-sonnet-4-5; anthropic/claude-sonnet-4-5-20250929; claude-sonnet-4-5; claude-sonnet-4-5-20250929; openrouter/anthropic/claude-sonnet-4.5.
- **DeepSeek R1**: DeepSeek reasoning model. Aliases: deepseek-ai/DeepSeek-R1; deepseek/deepseek-r1; openrouter/deepseek/deepseek-r1; openrouter/deepseek/deepseek-r1-0528; together_ai/deepseek-ai/DeepSeek-R1.
- **DeepSeek V3 / Chat**: DeepSeek chat model family. Aliases: deepseek-ai/DeepSeek-V3; deepseek/deepseek-chat; openrouter/deepseek/deepseek-chat-v3-0324; openrouter/deepseek/deepseek-chat-v3.1; together_ai/deepseek-ai/DeepSeek-V3.
- **Gemini 2.0 Flash**: Google Gemini Flash model. Aliases: gemini-2.0-flash; gemini/gemini-2.0-flash; google/gemini-2.0-flash-001.
- **Gemini 2.5 Pro Preview**: Google Gemini Pro preview model. Aliases: gemini-2.5-pro-preview-03-25; gemini/gemini-2.5-pro-preview-03-25.
- **Gemini 3 Pro Preview**: Google Gemini Pro preview model. Alias: google/gemini-3-pro-preview.
- **GPT-4.1**: OpenAI GPT-4.1 model. Aliases: gpt-4.1; gpt-4.1-2025-04-14; openai/gpt-4.1-2025-04-14.
- **GPT-4o**: OpenAI GPT-4o model. Aliases: gpt-4o; gpt-4o-2024-11-20.
- **GPT-5**: OpenAI GPT-5 model. Aliases: gpt-5; gpt-5-2025-08-07; openai/gpt-5; openai/gpt-5-2025-08-07.
- **GPT-OSS 120B**: OpenAI open-weight GPT-OSS model. Alias: openrouter/openai/gpt-oss-120b.
- **o1**: OpenAI reasoning model. Alias: o1.
- **o3**: OpenAI reasoning model. Aliases: o3-2025-04-03; o3-2025-04-16; openai/o3-2025-04-16.
- **o3-mini**: OpenAI smaller reasoning model. Aliases: o3-mini; o3-mini-2025-01-31; openai/o3-mini-2025-01-31.
- **o4-mini**: OpenAI smaller reasoning model. Aliases: o4-mini-2025-04-16; openai/o4-mini-2025-04-16.

B.3 Reasoning-Effort Normalization

Reasoning-effort labels in raw run names are normalized onto the model-configuration side. For direct providers that support the parameter, HAL Generalist passes reasoning_effort to the model call. For OpenRouter-backed runs, the same setting is mapped to a reasoning-token budget. The normalized harness remains unchanged.

Raw label pattern	Normalized harness	Normalized model configuration
HAL Generalist High Reasoning with gpt-5	HAL Generalist	GPT-5 with high reasoning effort
HAL Generalist Minimal Reasoning with gpt-5	HAL Generalist	GPT-5 with minimal reasoning effort
HAL Generalist No Reasoning with Claude Opus 4.1	HAL Generalist	Claude Opus 4.1 with no explicit reasoning budget
HAL Generalist Agent o4-mini high/low	HAL Generalist	o4-mini with high/low reasoning effort
HAL Generalist Agent o3 medium	HAL Generalist	o3 with medium reasoning effort

Table 6: Examples of reasoning-effort normalization. Reasoning settings are treated as model-call configuration, not as different HAL Generalist harnesses.

B.4 Semantic Harness and Scaffold Groups

The 68 recorded harness/scaffold labels collapse to 15 semantic harness groups. Where a raw scaffold label includes a model name or reasoning setting, the normalized harness removes that suffix and relies on the separate model/model-configuration field for the crossing. The joinable normalization table used for analysis is stored in `irt_data/harness_metadata.csv`; it preserves each raw scaffold label, records the tool-exposure mechanism and exposed-tool inventory, records embedded model and reasoning-effort hints when they appear in the scaffold label, and records curation notes such as the historical `My Agent` alias. The one-row-per-normalized-harness affordance flags are additionally stored in `irt_data/harness_affordances.csv` and mirrored onto `harness_metadata.csv` for raw-scaffold joins.

- **AssistantBench Browser Agent**: AssistantBench browser-agent scaffold. Alias: Assistantbench Browser Agent.
- **Browser-Use**: Browser-use web navigation scaffold. Aliases: Browser-Use; Browser-Use_test.
- **Claude Code**: Claude Code software-engineering scaffold. Alias: Claude Code.
- **ColBench Example Agent**: ColBench programming scaffold. Aliases: colbench_backend_programming colbench_example_agent; colbench_example_agent_gpt41.
- **CORE-Agent**: CORE-Bench problem-solving scaffold. Aliases: CORE-Agent; CORE-Agent(Claude-Sonnet-4-5); Core Agent; Core Agent Opus 4.5; Core Agent Opus 4.5 high; Core-Agent; Coreagent; coreagent.
- **HAL Generalist Agent**: General-purpose HAL scaffold. Reasoning-effort variants are model configurations, not separate harnesses. Aliases: HAL Generalist; HAL Generalist Agent; HAL Generalist High Reasoning; HAL Generalist Minimal Reasoning; HAL Generalist No Reasoning; HAL-Generalist-Agent; Hal Generalist Agent; Hal Generalist Agent Opus 4.5; Hal Generalist Agent Opus 4.5 high; hal_generalist_agent_deepseekaideepseekr1; hal_generalist_agent_deepseekaideepseekv3; hal_generalist_agent_gemini20flash; hal_generalist_agent_o3mini20250131_high; hal_generalist_agent_o3mini20250131_low.
- **HF Open Deep Research**: Open Deep Research scaffold for research-style tasks. Alias: HF Open Deep Research.
- **SAB Self-Debug**: ScienceAgentBench self-debug scaffold. Aliases: SAB Self-Debug Claude-3-7; SAB Self-Debug Claude-3-7 high; SAB Self-Debug Claude-3-7 low; SAB Self-Debug Claude-Haiku-4-5; SAB Self-Debug Claude-Haiku-4-5 High; SAB Self-Debug Claude-Opus-4-1; SAB Self-Debug Claude-Opus-4-1-High; SAB Self-Debug Claude-Sonnet-4-5; SAB Self-Debug Claude-Sonnet-4-5 High; SAB Self-Debug DS-V3; SAB Self-Debug GPT-5-Medium; SAB Self-Debug gemini-2-0-flash; SAB Self-Debug gemini-2-5-pro; SAB Self-Debug o3 medium; SAB Self-Debug o4-mini high; SAB Self-Debug o4-mini low; sab_selfdebug_dsr1; sab_selfdebug_gpt_41.
- **SciCode Tool Calling Agent**: SciCode tool-calling scaffold. Aliases: SciCode Tool Calling Agent; Scicode Tool Calling Agent.
- **SciCode Zero Shot Agent**: SciCode zero-shot scaffold. Alias: Scicode Zero Shot Agent.
- **SeeAct**: SeeAct web interaction scaffold. Alias: SeeAct.
- **SWE-Agent**: SWE-Agent software-engineering scaffold. Aliases: SWE-Agent; My Agent. The `My Agent` raw label is retained in metadata as a curation flag; trace metadata identifies these runs as `agents/SWE-agent-v1.0` filemap/simple-review configs.

- **TAU-bench FewShot**: TAU-bench few-shot scaffold. Aliases: TAU-bench FewShot; TauBench Few Shot; TauBench Few Shot High Reasoning; TauBench Few-Shot High Reasoning; TauBench Few-Shot Minimal Reasoning; TauBench FewShot High Reasoning; TauBench FewShot No Reasoning; taubench_fewshot_o3mini20250131_high.
- **TAU-bench ToolCalling**: TAU-bench tool-calling scaffold. Alias: Taubench ToolCalling.
- **USACO Episodic + Semantic**: USACO coding scaffold. Aliases: USACO Episodic + Semantic; usaco_episodic__semantic_deepseekaideepseekr1; usaco_episodic__semantic_deepseekaideepseekv3; usaco_episodic__semantic_gpt4120250414; usaco_episodic__semantic_o3mini20250131_high; usaco_episodic__semantic_o3mini20250131_low.

C Tool-Exposure Taxonomy

This appendix records how environment affordances are passed through to the underlying model for each normalized harness group in Appendix B. The unit here is the normalized harness, not the raw run label: reasoning-effort variants and model-name suffixes remain model-configuration metadata, not new tool-exposure mechanisms. The audit uses the local agent source when available and trace/config metadata where the source for a historical scaffold is not present in this repository.

C.1 Mechanism Index

- **Native tool schema** (2 harnesses): TAU-bench FewShot; TAU-bench ToolCalling.
- **Code-agent tool registry** (4 harnesses): CORE-Agent; HAL Generalist Agent; HF Open Deep Research; SciCode Tool Calling Agent.
- **Text action protocol** (2 harnesses): Claude Code; SWE-Agent.
- **Framework-mediated browser control** (3 harnesses): AssistantBench Browser Agent; Browser-Use; SeeAct.
- **Harness-side augmentation** (3 harnesses): ColBench Example Agent; SAB Self-Debug; USACO Episodic + Semantic.
- **No explicit tools** (1 harness): SciCode Zero Shot Agent.

C.2 Exposed-Tool Inventory

The joinable source for this inventory is `irt_data/harness_metadata.csv`, in the `exposed_tools` and `exposed_tools_note` columns. The entries below use normalized harness labels and summarize the model-visible tools or, for harness-side augmentation, the harness actions that shape the trace without becoming callable model tools. The derived binary affordance flags, such as code execution, shell access, web search, browser control, native domain tools, retrieval context, external evaluation, and self-debug feedback, are stored in `irt_data/harness_affordances.csv`.

- **AssistantBench Browser Agent**: browser navigation, click, type, scroll, page extraction, and done/answer actions mediated by `browser_use.Agent`.
- **Browser-Use**: browser navigation, click, type, scroll, page extraction, and done/answer actions from the Browser-Use framework.
- **Claude Code**: shell command, file read/search/edit, test-command, and submit/finish actions through the external CLI text protocol.
- **ColBench Example Agent**: harness dialogue and final-answer submission; no callable environment tools are exposed to the model.
- **CORE-Agent**: `web_search`, page visit, `python_interpreter`, `execute_bash`, `inspect_file_as_text`, `edit_file`, `file_content_search`, `query_vision_language_model`, and `final_answer`.
- **HAL Generalist Agent**: the CORE-style tools `web_search`, page visit, `python_interpreter`, `execute_bash`, `inspect_file_as_text`, `edit_file`, `file_content_search`, and `query_vision_language_model`, plus benchmark-dependent adapters such as TAU-bench airline action wrappers, AppWorld API execution, and ColBench `ask_user`/`finish_task`.
- **HF Open Deep Research**: `visualizer`, `inspect_file_as_text`, a managed `search_agent`, `web_search`, `visit_page`, `page_up`, `page_down`, `find_on_page_ctrl_f`, `find_next`, and `find_archived_url`.

- **SAB Self-Debug**: complete-program generation, harness code execution, and self-debug feedback. These are harness-side stages, not model-callable tools.
- **SciCode Tool Calling Agent**: `web_search`, `python_interpreter`, `wikipedia_search`, and `final_answer`.
- **SciCode Zero Shot Agent**: no explicit tools.
- **SeeAct**: browser observation, click, type, select, scroll, navigate, and stop/answer actions from the SeeAct browser-action protocol.
- **SWE-Agent**: `bash`, `str_replace_editor` view/create/replace/insert/undo operations, `filemap`, `submit`, and diff-state context.
- **TAU-bench FewShot**: airline-native tools `book_reservation`, `calculate`, `cancel_reservation`, `get_reservation_details`, `get_user_details`, `list_all_airports`, `search_direct_flight`, `search_onestop_flight`, `send_certificate`, `think`, `transfer_to_human_agents`, `update_reservation_baggages`, `update_reservation_flights`, and `update_reservation_passengers`.
- **TAU-bench ToolCalling**: the same airline-native tools as TAU-bench FewShot, passed through the native `tools_info` schema.
- **USACO Episodic + Semantic**: retrieval prompts, episodic and semantic examples, and judge execution as harness-side stages; no interactive callable environment tools are exposed to the model.

C.3 Canonical Tool-Passage Mechanisms

Native tool schema. The benchmark or harness passes OpenAI-style tool/function schemas to the model API or to a framework that preserves those schemas as the model-visible contract. The model returns structured tool calls, and the harness executes them. This category is reserved for cases where the environment's own schema, such as `isolated_env.tools_info`, is what the model is asked to call.

Code-agent tool registry. Python callables are registered as tools in a code-agent runtime, most often a `smolagents.CodeAgent`. The model sees a tool catalogue through the agent runtime and emits executable code or tool-use steps that call those Python tools. Some registered tools are thin adapters over benchmark actions, but the model-visible schema is the wrapper function signature and docstring, not the benchmark's native tool schema. Thus HAL Generalist on TAU-bench is classified here: its wrappers eventually call `isolated_env.step(Action(...))`, but the original `tools_info` schema is not passed through directly to the model.

Text action protocol. Tools are described in the prompt as text and the model is instructed to emit a parseable action, command, or JSON block. The harness parses that text and maps it to an environment action.

Framework-mediated browser control. A browser-agent library supplies an action space such as click, type, scroll, navigation, and page observation. The benchmark environment is not passed as domain-specific tool schemas; instead, the model acts through the browser-control framework.

Harness-side augmentation. Retrieval, self-debugging, code execution, or evaluation happens outside the model-visible action interface. The model may see retrieved context or error messages, but it is not given callable environment tools for that benchmark step.

No explicit tools. The scaffold is a direct prompt-to-answer or prompt-to-code generator for the relevant benchmark. Any execution or grading is outside the model's callable interface.

Harness-side augmentation spans several strengths. At the light end, a harness may insert retrieved examples or generated context into a prompt before a normal model call. At the middle, it may run an external environment loop and feed the model textual observations without giving it callable tools. At the heavy end, it may execute generated programs, collect runtime errors, and ask the model to debug in subsequent turns. In all three cases the augmentation can materially change behavior, but it is not an environment tool interface exposed to the model.

C.4 Normalized Harness Labels

- **AssistantBench Browser Agent**: framework-mediated browser control. Uses the `browser_use.Agent` stack with LangChain chat models and a browser object; browser affordances are mediated by the browser-use framework rather than benchmark-specific schemas.
- **Browser-Use**: framework-mediated browser control. Historical online_mind2web Browser-Use runs are recorded as browser-use scaffolds. Local source for this exact scaffold is absent, so the label is assigned from trace configuration and the Browser-Use harness family.
- **Claude Code**: text action protocol / external CLI scaffold. Claude Code runs are software-engineering agents whose environment actions are exposed through the CLI/scaffold command interface, not through benchmark-native tool schemas.
- **ColBench Example Agent**: harness-side augmentation, medium strength. The ColBench scaffold calls the model for textual actions in a human-interaction environment. Environment stepping is mediated by the harness dialogue loop rather than by callable model tools.
- **CORE-Agent**: code-agent tool registry. CORE-Agent is implemented as a `smolagents.CodeAgent` with core Python tools such as web/search, page visit, Python execution, bash execution, file inspection/editing, and final answer.
- **HAL Generalist Agent**: code-agent tool registry with benchmark adapters. The general path uses `smolagents.CodeAgent` and `CORE_TOOLS`. Some benchmarks add adapter tools: TAU-bench actions are hard-coded wrappers around `isolated_env.step`, while AppWorld-style APIs can be converted from API docs into smolagents tools.
- **HF Open Deep Research**: code-agent tool registry. The Open Deep Research scaffold constructs a smolagents research agent with web, browser, visualizer, and file-inspection tools. Tools are exposed through the smolagents runtime.
- **SAB Self-Debug**: harness-side augmentation, heavy strength. The ScienceAgentBench self-debug scaffold asks the model to write complete Python programs, then the harness executes/debugs externally and may feed error messages back. The execution environment is not exposed as callable model tools.
- **SciCode Tool Calling Agent**: code-agent tool registry. Uses a smolagents `CodeAgent` with web search, Python interpreter, Wikipedia search, and final-answer tools. These are general scaffold tools rather than SciCode benchmark-native APIs.
- **SciCode Zero Shot Agent**: no explicit tools. Generates code from prompts with direct model calls. There is no callable tool interface exposed to the model in this scaffold.
- **SeeAct**: framework-mediated browser control. Historical online_mind2web SeeAct runs are browser-interaction scaffolds. Local source is not present in this checkout; classification is based on trace/run metadata and the SeeAct harness family.
- **SWE-Agent**: text action protocol / external CLI scaffold. SWE-Agent exposes repository operations through its command/tool configuration and command parser. The model interacts with the environment via scaffold commands rather than benchmark-native tool schemas. The historical raw label `My Agent` is normalized into this group because its trace metadata points to `agents/SWE-agent-v1.0` filemap/simple-review configurations; the raw label is retained in `irt_data/harness_metadata.csv` for possible exclusion or data-cleaning sensitivity checks.
- **TAU-bench FewShot**: native tool schema. The TAU-bench few-shot harness passes `isolated_env.tools_info` and the environment wiki to `FewShotToolCallingAgent`; model actions are structured TauBench tool calls executed by the environment.
- **TAU-bench ToolCalling**: native tool schema. The TAU-bench tool-calling harness passes `isolated_env.tools_info` to `ToolCallingAgent`. Optional perturbations modify the schema and responses but preserve the native tool-call path.
- **USACO Episodic + Semantic**: harness-side augmentation, light to medium strength. The USACO scaffold performs solve and retrieval stages in the harness. Retrieved examples or context are inserted into prompts; the model is not given an interactive callable tool interface for the benchmark environment.

C.5 Tool and Affordance Metadata Construction

The harness metadata is intentionally split into three related layers: `tool_exposure_mechanism`, `exposed_tools`, and binary `affordance_*` flags. The first layer records how tools or environment actions are passed to the model. The second records the concrete tool names or harness-side stages associated with each normalized harness. The third collapses those tool lists into coarse feature flags that can be joined onto outcome and trace tables.

Harness normalization. Raw scaffold labels are first mapped to the normalized harness labels in Appendix B. This step removes provider/model suffixes and reasoning-setting suffixes from scaffold names, while preserving the raw label in `raw_scaffold`. For example, historical `My Agent` runs are normalized to **SWE-Agent**, with the original label retained for possible sensitivity analysis. The normalized harness label is then used as the unit for mechanism labels, exposed-tool inventories, and affordance flags.

Tool extraction. For harnesses with local source, exposed tools are extracted from the agent or benchmark code. Smolagents-based harnesses are read from their `CodeAgent` or `ToolCallingAgent` construction sites and tool classes. This yields concrete lists such as `web_search`, `python_interpreter`, `execute_bash`, `inspect_file_as_text`, `edit_file`, and `file_content_search`. TAU-bench tools are taken from the environment-native `isolated_env.tools_info` path and the observed airline action names. SWE-Agent tools are read from the filemap/simple-review configuration bundles, including shell, file-map, editor, submit, and diff-state actions. When the exact local source is absent, as for some historical browser scaffolds, the inventory is recorded at the framework-action level and the caveat is stored in `exposed_tools_note`.

Harness-side stages. For harness-side augmentation, `exposed_tools` records stages that shape the trace rather than model-callable tools. Thus SAB Self-Debug records complete-program generation, harness code execution, and self-debug feedback; USACO records retrieval/example stages and judge execution; ColBench records harness dialogue and final-answer submission. These entries are useful for affordance analysis but should not be interpreted as native or registered model tools.

Affordance flags. The derived `affordance_*` columns are stored once per normalized harness in `irt_data/harness_affordances.csv` and mirrored onto `irt_data/harness_metadata.csv`. The flags are:

- `affordance_code_generation`
- `affordance_code_execution`
- `affordance_shell_access`
- `affordance_file_read`
- `affordance_file_edit`
- `affordance_web_search`
- `affordance_web_page_visit`
- `affordance_browser_control`
- `affordance_vision`
- `affordance_native_domain_tools`
- `affordance_benchmark_api_tools`
- `affordance_human_dialogue`
- `affordance_retrieval_context`
- `affordance_external_judge_or_eval`
- `affordance_self_debug_feedback`
- `affordance_complete_program_prompt`

A flag is true when that capability is available through the harness or its evaluation loop, regardless of whether the capability is exposed as a native tool schema, a code-agent registered function, a text command, or a harness-side augmentation.

Interpretation. These affordance flags intentionally abstract away from the passage mechanism. For example, Claude Code and SAB Self-Debug both have code execution or debugging affordances, but Claude Code exposes execution through an interactive agent scaffold whereas SAB Self-Debug executes generated programs in the benchmark harness and may feed errors back into later prompts. Conversely, TAU-bench ToolCalling and HAL Generalist can both execute airline domain actions, but

the former passes native schemas while the latter re-exports some actions through smolagents wrappers. Analyses should therefore use affordance flags together with `tool_exposure_mechanism`, not as replacements for the mechanism taxonomy.

C.6 Interpretation for Trace Features

This metadata separates *available affordances* from *model-visible tool passage*. For example, TAU-bench ToolCalling and HAL Generalist can both execute `update_reservation_flights`, but the former passes the TauBench schema directly through `tools_info`, whereas the latter re-exports that action as a hand-written smolagents wrapper. Likewise, a browser scaffold and a code-agent scaffold may both access the web, but one does so through page-control actions and the other through search/page tools. Downstream trace features should therefore include both coarse affordance flags (for example browser control, code execution, web search, file editing) and this tool-passage mechanism label.

D SWE-bench Trace Audit: Registry vs. Repo Workflow

HAL Generalist and SWE-Agent both expose code-generation, code-execution, shell, file-read, and file-edit affordances on SWE-bench. In HAL Generalist, these affordances pass through a smolagents-style code-agent registry with tools such as `python_interpreter`, `execute_bash`, `inspect_file_as_text`, `edit_file`, `file_content_search`, and `final_answer`. SWE-Agent exposes a repo-repair workflow with a specialized action protocol, persistent editor semantics, file-map context, diff/review state, and an explicit submit action. The audit below therefore focuses on the model-facing workflow difference, rather than treating live execution or editing as the distinguishing factor.

To make this distinction concrete, we audited matched GPT-5 traces on SWE-bench Verified Mini. The HAL Generalist run `swebench_verified_mini_hal_generalist_gpt520250807_1755463923` and the SWE-Agent run `swebench_verified_mini_sweagentgpt520250807_1754592641` cover the same 50 tasks. HAL Generalist fails while SWE-Agent succeeds on 26 of these tasks. The joinable extraction artifacts are stored in `irt_data/trace_analysis/swebench_hal_vs_sweagent/`: the full candidate set is `gpt5_hal_fail_swe_success_tasks.csv`, and the paired trace examples below are `gpt5_trace_examples.csv`.

The examples suggest a concrete follow-up ablation: keep the same base model and sandbox budget, but vary only the repo-repair interface. For HAL Generalist, this could mean adding a SWE-Agent-like file-map/edit/review/submit layer while preserving its underlying execution tools. For SWE-Agent, this could mean disabling file-map or diff-state context while leaving shell and editor access intact. Such paired ablations would separate the value of live execution from the value of repo-specialized workflow structure.

Task	Issue	HAL Generalist failed trace	SWE-Agent succeeded trace
Django 12143	Regex escaping in Django admin formset prefixes	8 LLM calls; uses registry tools including `file_content_search`, `edit_file`, and `execute_bash`; final message says the referenced upstream code is absent and no patch is generated.	25 LLM calls; uses `bash`, `str_replace_editor`, and `submit`; final message reports locating the ModelAdmin edited-PK helper, applying `re.escape`, and verifying with a reproduction script.
Django 12050	Preserve list-vs-tuple type in query lookup resolution	19 LLM calls; uses search, edit, bash, and a Python interpreter; final message is still in patch-construction mode and the submitted result fails grading.	16 LLM calls; uses repository shell/editor actions and submit; trace records a plan to inspect ORM code, reproduce the coercion bug, patch it, and rerun the reproduction script.
Sphinx 9698	Remove parentheses from Sphinx Python-property index entries	19 LLM calls; uses repeated registry searches and edits; final message returns a patch description, but the benchmark marks it unresolved.	42 LLM calls; uses repeated shell/editor operations and submit; final message reports a minimal change in the Sphinx Python-domain implementation.

Table 7: Matched GPT-5 examples where HAL Generalist fails and SWE-Agent succeeds on the same SWE-bench Verified Mini task. Both harnesses can execute and edit code; the visible difference is the model-facing workflow: a broad registry of general tools versus a repo-specific repair protocol.